

Módulo 5 | *Coderhouse*

Domina el SQL Avanzado para Análisis de Datos

CTEs, Funciones de Ventana y la lógica dinámica
dine transforma consultas complejas en
código escalable.

Los 3 Superpoderes del Analista de Datos

01.

Organización y Limpieza (CTEs)

¿Cómo dejamos de escribir código espagueti y creamos consultas que se leen de forma modular?

02.

Cálculos sin Colapsar (Window Functions)

¿Cómo calculamos tendencias, rankings y acumulados sin perder el detalle de cada registro individual?

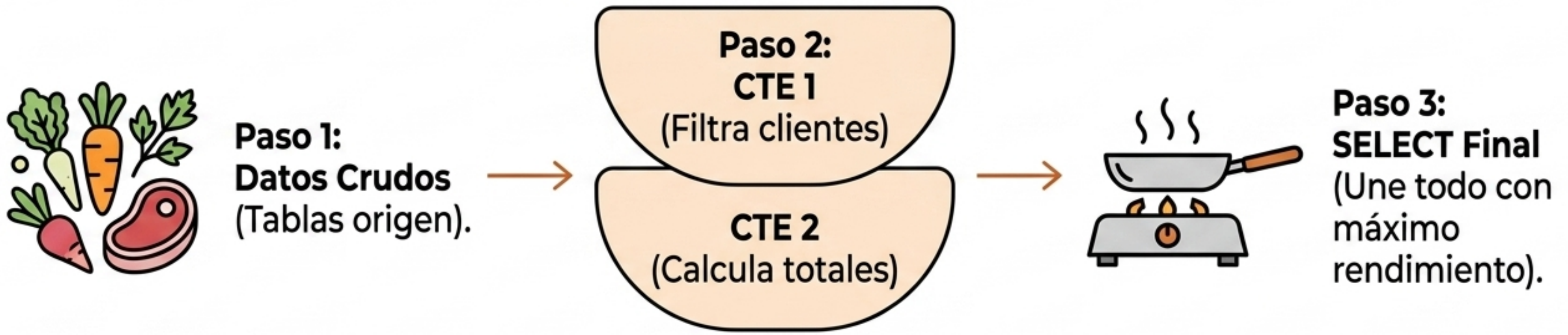
03.

Lógica de Negocio (Fechas y CASE)

¿Cómo inyectamos decisiones dinámicas y cortes temporales exactos en nuestros reportes?

CTEs: La 'Mise en Place' de tu Código

Expresiones de Tabla Comunes (WITH) para filtrar y pre-agregar antes del plato final.



Son tablas virtuales en memoria que se leen intuitivamente de arriba hacia abajo.

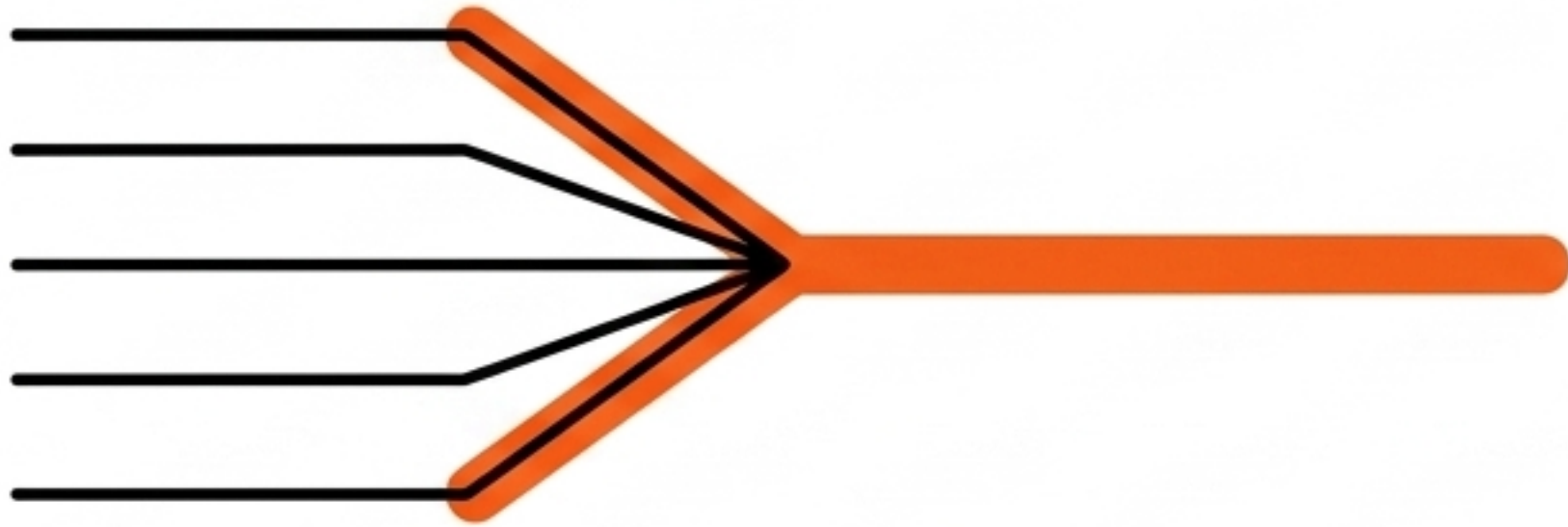
Coder Tips

Evita errores de sintaxis: Usa WITH solo una vez al principio. Cierra siempre tus CTEs entre paréntesis. ¡Nunca olvides asignar un ALIAS a tus columnas calculadas o no podrás llamarlas al final!

Funciones de Ventana: Agregación sin destrucción

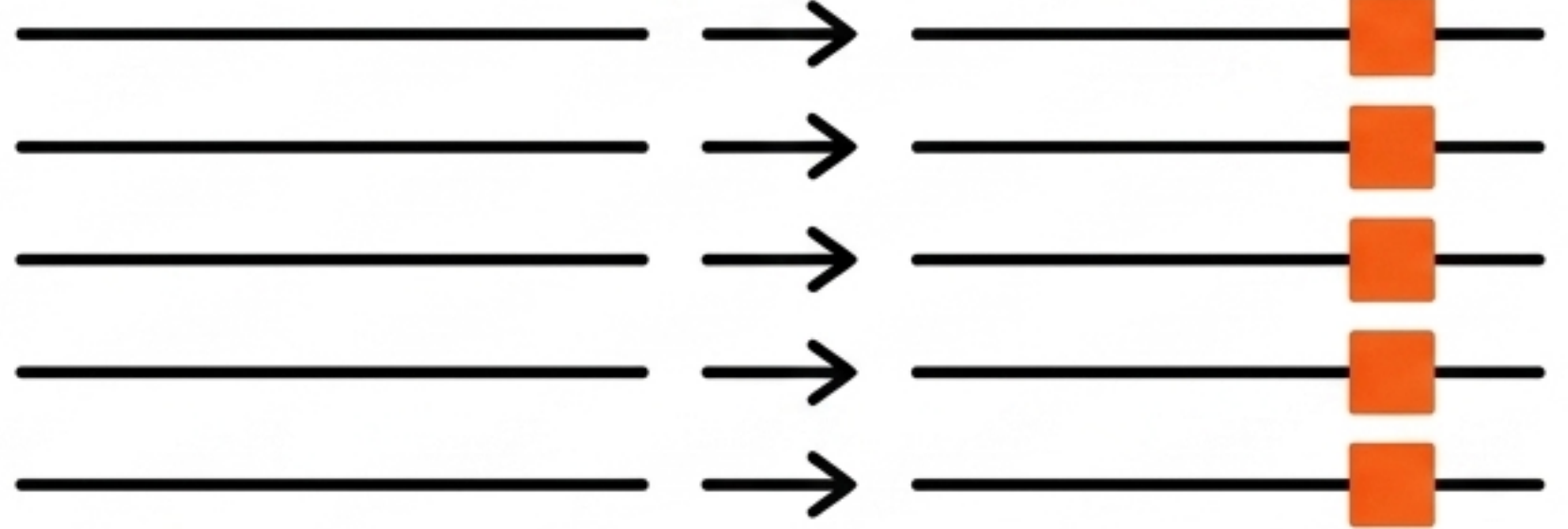
Realiza cálculos sobre grupos de filas... **sin aplanar la tabla original.**

GROUP BY



Colapsa los datos. Pierdes el detalle de cada transacción para ver el Total.

OVER y PARTITION BY



Conserva el registro individual e inyecta la métrica grupal al lado. La magia ocurre con PARTITION BY, que le indica a la función dónde reiniciar su contador.

¿Qué pasa cuando hay un empate métrico?

Escenario de ventas: Carlos (100 pts), Beatriz (100 pts), David (90 pts).

ROW_NUMBER()

Ignora empates. Asigna secuencial único.

1, 2, 3.

RANK()

Permite empates, pero salta las posiciones posteriores.

1, 1, 3.

DENSE_RANK()

Permite empates y mantiene la secuencia compacta.

1, 1, 2.



Ideal para buscar los Top N niveles reales

Cortando el pastel y escalando tendencias

NTILE(n)



Divide las filas en n grupos lo más equitativos posible. Ideal para calcular Cuartiles NTILE(4) o Deciles NTILE(10).

Si sobran filas, se asignan a los primeros grupos.
Ej: 6 registros en 4 grupos = 2, 2, 1, 1.

Running Totals



`SUM(monto) OVER (ORDER BY fecha)`

Permite ver la tendencia progresiva de ingresos a lo largo del tiempo.



¡Cuidado! Si olvidas el ORDER BY, obtendrás la suma global repetida en cada fila, no el acumulado.

Viajando en el tiempo de los datos: LAG() y LEAD()

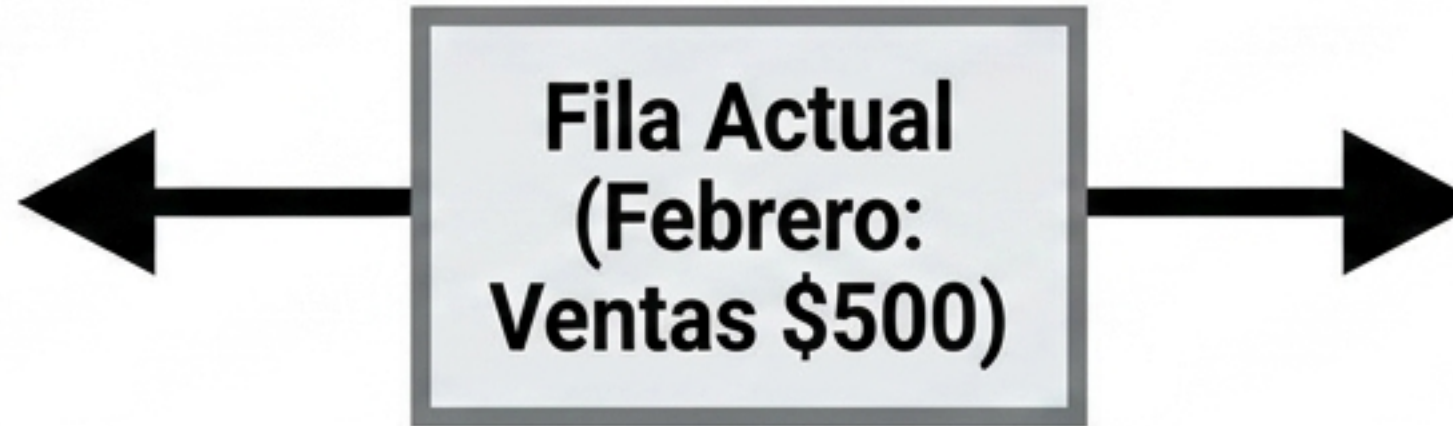


LAG()

Trae el valor de la fila anterior (Enero: \$400).

Vital para calcular crecimientos Month over Month.

Devuelve NULL en el extremo inicial.



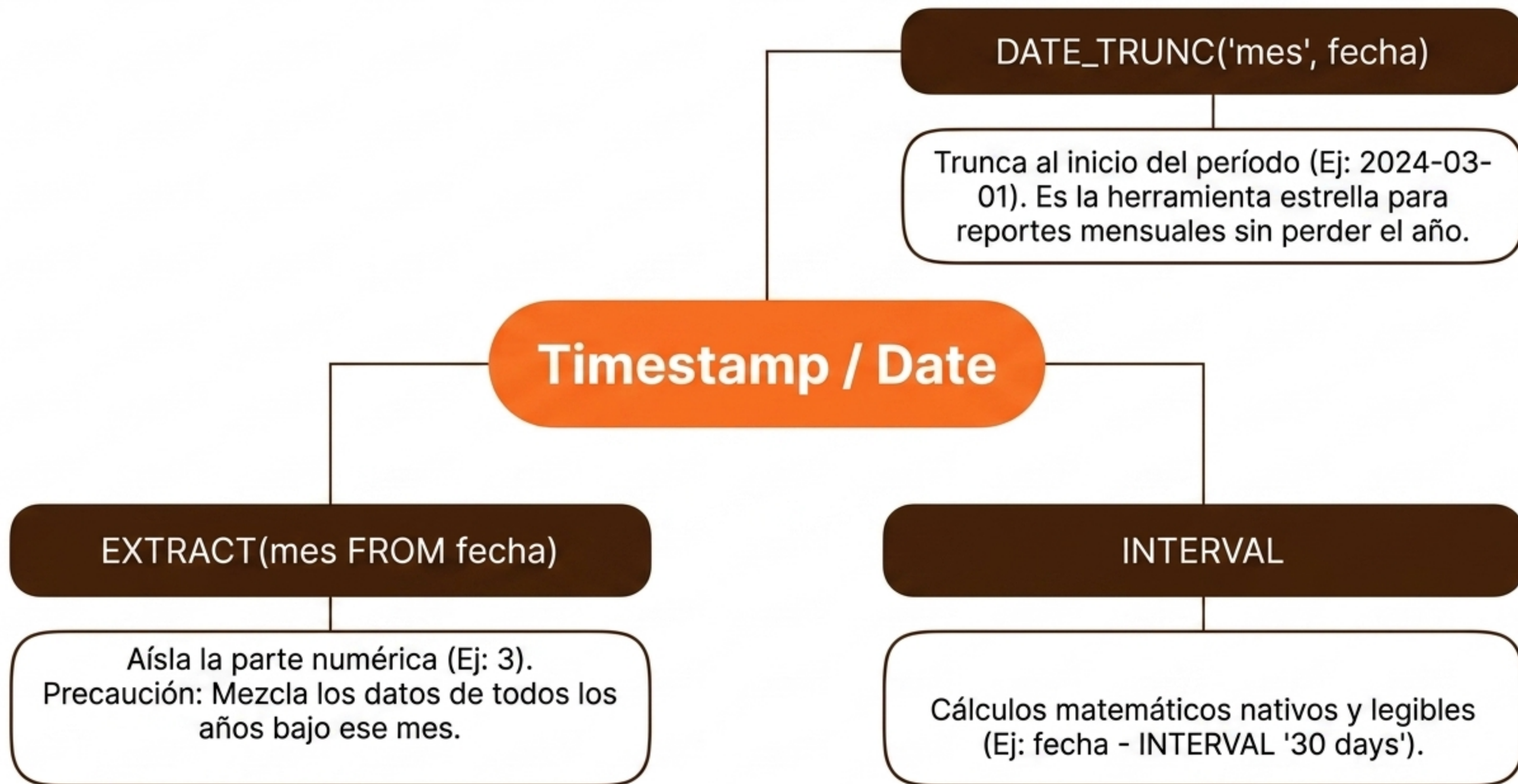
LEAD()

Trae el valor de la fila siguiente (Marzo: \$600).

Ideal para medir días transcurridos hasta la próxima compra.

Devuelve NULL en el extremo final.

Dominando el Espacio-Tiempo (Fechas)



Inyectando Lógica de Negocio: CASE WHEN

Transforma reglas empresariales dinámicas en columnas de SQL estructuradas.

01. La Regla de Especificidad

Se evalúa de arriba hacia abajo y frena en la primera condición verdadera (THEN). Programa siempre desde lo más específico hacia lo más general.

02. El Salvavidas 'ELSE'

Nunca olvides el ELSE. Si lo omites y nada se cumple, SQL devolverá NULL, sesgando permanentemente las agrupaciones de tus reportes.

03. Consistencia de Tipos

Todos los resultados que retornes deben ser del mismo tipo de dato. ¡No mezcles números enteros con cadenas de texto en la misma columna!

Tu Toolkit 4.0 está completo.

Tienes el poder de modelar la realidad del negocio desde la base de datos.

Estructura: CTEs (WITH) para legibilidad y modularidad.

Profundidad: Window Functions (OVER(), PARTITION BY, RANK, LAG) para cálculos sin colapsar el detalle.

Adaptabilidad: Fechas (DATE_TRUNC) y Lógica (CASE WHEN) para reportes inteligentes.